

maquette-scad

Parametric CAD in Typst, via OpenSCAD + Manifold

github.com/bernsteining/maquette · bernsteining.github.io/maquette



Version 0.1.0 · August 27, 2026

CONTENTS

Contents

Introduction	3
Where to find sample .scad files	4
Quickstart	4
Building geometry from Typst — scadypst	5
DSL reference	6
3D primitives	6
2D primitives (extrusion sources)	6
2D → 3D lifting (and 3D → 2D)	6
Transforms (2D or 3D)	6
Booleans and hulls	6
Colours + alpha	7
scadypst() — the compile call	8
Name clashes with Typst built-ins	9
Compiling .scad sources — compile-scad	10
Real-world showcase — Cyclone-PCB-Factory	11
Language coverage	12
Design notes	13

Introduction

maquette-scad is a Typst plugin that turns OpenSCAD-flavored geometry into meshes for maquette to render. Two entry points, both returning PLY bytes you feed to `render-ply`:

- **scadypst(tree)** — build geometry with Typst expressions (`cube`, `sphere`, `difference`, `translate`, ...). The tree is a plain dict of dicts you compose with Typst's own `for` / `range` / `calc`.
- **compile-scad(source, files?, bin?, font?, fn?, trace?)** — hand in the text of an existing `.scad` file. Full OpenSCAD language surface: `function`, `module`, `for`, list comprehensions, `include` / `use`, `$fn` / `$fa` / `$fs`.

The CSG kernel is Manifold — every boolean is guaranteed watertight, no BSP-style open faces on non-trivial cuts.

This doc covers the maquette-scad **API**: the Typst-side entry points, their options, and the patterns for using them. It does **not** teach OpenSCAD — for that see the OpenSCAD Users Manual (language) or `crates/maquette-scad/maquette-scad/maquette-scad.typ` (per-function docstrings for every DSL helper). Once you have PLY bytes, downstream rendering (camera, lighting, materials, shadows, tone mapping) is maquette's job — see maquette's documentation. See `crates/maquette-scad/FIDELITY.md` for the exhaustive OpenSCAD language coverage tracker.

Where to find sample .scad files

The DSL examples below run inline — no external files needed. To exercise the .scad ingestion path (`compile-scad(read(...))`), grab source from:

- `openscad/openscad examples/` — the official example set that ships with the OpenSCAD editor.
- BOSL2 — comprehensive OpenSCAD utility library with hundreds of documented example fragments.
- Thingiverse (`openscad` tag) — large community archive; many things ship the .scad source alongside the .stl.

Quickstart

Two entry points, one plugin. Route the returned PLY to `render-ply` with whatever config you'd give any other mesh.

```
#import "@preview/maquette-scad:0.1.0": scadypst, cube, sphere, difference
#import "@preview/maquette:0.1.3": render-ply

#let part = scadypst(
  difference(
    cube(20, center: true),
    sphere(12, fn: 48),
  )
)
#render-ply(part, camera: (40, 40, 40), up: (0, 0, 1))
```

`compile-scad` is the same shape but takes source text:

```
#import "@preview/maquette-scad:0.1.0": compile-scad
#import "@preview/maquette:0.1.3": render-ply

#let part = compile-scad(read("model.scad"))
#render-ply(part)
```

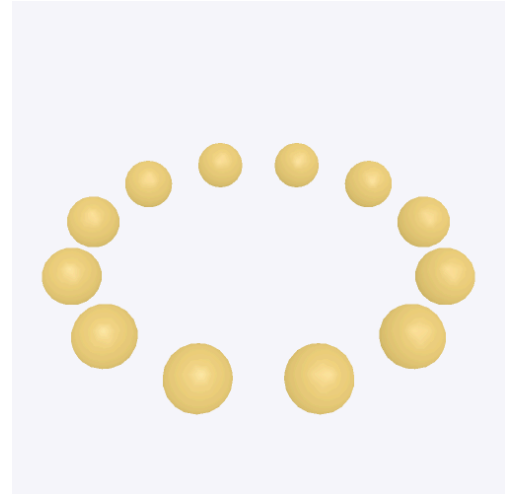
Below, `show-part` is a thin wrapper this doc uses so examples can focus on the geometry without repeating render config. In real use you'd inline the `render-ply(bytes, ..)` call.

Building geometry from Typst — scadypst

`scadypst(tree)` compiles a tree of Typst dicts into PLY bytes. The dicts come from helpers imported from the plugin (`cube` / `sphere` / `translate` / `difference` / `hull` / `linear-extrude` / ...); each helper **builds** a node in the tree, nothing runs until `scadypst` sees the whole thing.

The value of the DSL over `compile-scad(read("part.scad"))` is Typst integration: because the tree is Typst code, you get Typst's own procedural facilities for free. Here a `..for` spread inside `union` places twelve spheres on a ring:

```
#let ring = union(..for i in range(12) {  
  let a = i * 30  
  let r = 20  
  (translate(  
    (calc.cos(a * 1deg) * r, calc.sin(a * 1deg) * r, 0),  
    sphere(3, fn: 24),  
  ),)  
})  
#show-part(scadypst(ring))
```



The equivalent OpenSCAD for `(i = [0:11]) { ... }` works too via `compile-scad` — but a new-in-Typst design will usually reach for Typst's iteration first, since it composes cleanly with variadic ops (`union` / `difference` / `hull` / `intersection`) via the `..` spread syntax and reads with the rest of your document code.

DSL reference

Every helper takes Typst-native named arguments (`cube(20, center: true)` instead of OpenSCAD's `cube(20, center=true);`). Signatures below list positional args first, then named. `fn: N` is the OpenSCAD \$fn per-primitive segment count.

3D primitives

- **cube(size, center: false)** — size is a number (uniform) or a 3-array (x, y, z).
- **sphere(r, fn: none)**
- **cylinder(h, r: none, r1: none, r2: none, center: false, fn: none)** — pass r for a straight cylinder, r1 + r2 for a cone / frustum.
- **polyhedron(points, faces)** — raw mesh: points = ((x, y, z), ...), faces = ((i, j, k, ...), ...).

2D primitives (extrusion sources)

- **square(size, center: false), circle(r, fn: none), ellipse(w, h, fn: none).**
- **polygon(points, paths: none)** — arbitrary 2D shape. paths is a list of index-rings into points: first ring is the outer boundary, rest are holes.
- **ngon(sides, r, fn: none)** — regular polygon.
- **star(points, outer, inner)** — n-pointed star.
- **rounded-square(w, h, r, fn: none)** — rectangle with radius-r corners.
- **scad-text(str, size: 10)** — glyph outlines from the font passed to `scadypst()`'s font: param.
- **import-mesh(file)** — reference an STL/OBJ passed via `scadypst()`'s bin: dict.

2D → 3D lifting (and 3D → 2D)

- **linear-extrude(h, child, center: false, twist: 0, scale: 1, slices: none)** — extrude a 2D shape to a height h. twist is degrees over the full height; scale is a taper factor (0..1) or an (sx, sy) pair.
- **rotate-extrude(child, angle: 360, fn: none)** — revolve a 2D profile (living in the +x half-plane) around the z-axis.
- **projection(child)** — flatten a 3D solid to its 2D shadow on the Z=0 plane.

Transforms (2D or 3D)

- **translate(v, child)** — v is a 2- or 3-vector.
- **rotate(deg, child)** — Euler degrees, (x, y, z) or a single number (2D rotation).
- **scale(v, child)** — per-axis scale.
- **mirror(v, child)** — reflect across a plane whose normal is v.
- **multmatrix(m, child)** — 4×4 (or 4×3) affine matrix; escape hatch for anything the named transforms can't do.
- **resize(v, child)** — non-uniform scale to fit a target bounding box.
- **offset(d, child)** — grow (d > 0) or shrink (d < 0) a 2D shape by d.
- **color(rgb, child, alpha: none)** — RGB is a 3-array of 0..1 floats, or a 4-array (r, g, b, a). See the Colours + alpha section below.

Booleans and hulls

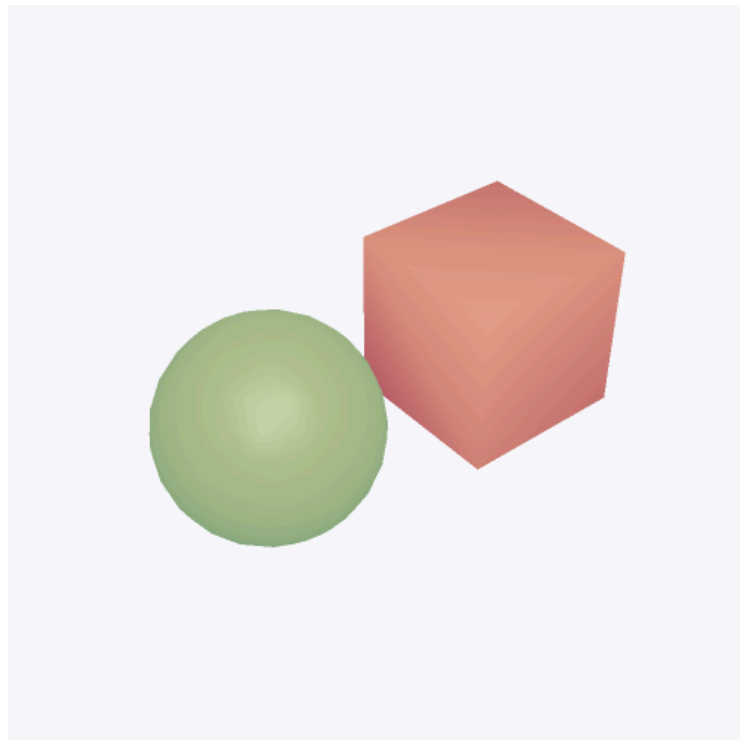
- **union(..items), difference(..items)** (first minus the rest), **intersection(..items).**
- **hull(..items)** — convex hull of the union (3D).
- **minkowski(..items)** — Minkowski sum (3D).

All variadic — use `..` spread with a Typst `for` block to feed a computed list of children.

Colours + alpha

Colours propagate through boolean ops; the emitted PLY carries per-vertex RGB and render-ply picks it up automatically. Every part in the tree without a `color()` wrapper inherits the render config's color: — the “everything else is yellow” look in the OpenSCAD-logo demo comes from that fallback.

```
#show-part(  
  scadypst(union(  
    color((0.9, 0.3, 0.3), cube(10, center: true)),  
    color((0.3, 0.7, 0.4), translate((15, 0, 0), sphere(6, fn:  
32))),  
  )),  
  color: none,  
)
```



Alpha ships end-to-end (scad → PLY → maquette rasterizer). Two forms — pick either:

```
color((0.9, 0.35, 0.35, 0.5), child)           // RGBA in one 4-array  
color((0.9, 0.35, 0.35), child, alpha: 0.5)    // RGB + separate alpha
```

Interior features show through translucent subtrees — this is how the OpenSCAD IDE renders `#` (highlight) modifiers, and how the demo's OpenSCAD logo shows the drilled holes and back of the sphere through the coloured cylinder.

scadypst() — the compile call

```
scadypst(node, bin: (:), font: none, fn: 32)
```

- **node** — the tree returned by any DSL helper. Usually the top-level union / difference / a single primitive.
- **bin** — dict of sidecar bytes referenced by `import-mesh(file)`. Keys match the string passed to `import-mesh`, values are `read("path", encoding: none)`.
- **font** — TTF/OTF bytes for `scad-text(...)`. One font per compile.
- **fn** — default `$fn` for the whole compile. Per-primitive `fn`: overrides.

Returns the same PLY bytes that `compile-scad` returns — pass it to `render-ply`.

Name clashes with Typst built-ins

Seven DSL helpers shadow Typst built-ins when you `#import "..": *`:

scale, rotate, circle, square, ellipse, polygon, color (plus text → renamed to scad-text at export).

Two ways around it:

```
// Option 1 – namespace the plugin
#import "@preview/maquette-scad:0.1.0"
#let part = maquette-scad.scadypst(
  maquette-scad.difference(
    maquette-scad.cube(20, center: true),
    maquette-scad.sphere(12, fn: 48),
  )
)

// Option 2 – glob-import the plugin's `scad-*` aliases
#import "@preview/maquette-scad:0.1.0": scad-color, scad-scale, scad-rotate, scad-circle, scad-square,
scad-ellipse, scad-polygon
```

Both preserve Typst's own `#text` / `#circle` / etc. Aliases live alongside the originals so you can mix as needed.

Compiling .scad sources — compile-scad

For standalone .scad files, `compile-scad(source_text)` is all you need:

```
#let part = compile-scad(read("part.scad"))
#render-ply(part)
```

Real-world .scad files rarely stand alone — they use `/ include` a library, import an STL/OBJ for a fastener, or ship a font for embossed text. The wasm sandbox has no filesystem, so sidecars are passed explicitly as bytes:

```
#compile-scad(read("main.scad"),
  files: (
    "utils.scad":      read("utils.scad"),
    "gears.scad":      read("gears.scad"),
    "MCAD/nuts_and_bolts.scad": read("MCAD/nuts_and_bolts.scad"),
  ),
  bin: (
    "mount.stl":      read("mount.stl", encoding: none),
  ),
  font: read("logo.ttf", encoding: none),
  fn: 48,
  trace: "trace.log",
)
```

Options:

- **source** (positional string) — the .scad text. `read("path.scad")` is the usual way to get it.
- **files** (dict, optional) — every use `<name> / include <name>` resolves against this dict. Keys are the exact names the .scad uses; nested paths (`MCAD/foo.scad`) are dict keys with slashes.
- **bin** (dict, optional) — sidecar mesh files referenced by `import()` in the source. Keys match the `import` argument. Decoded formats: STL and OBJ (DXF, 3MF and AMF are not).
- **font** (bytes, optional) — a TTF/OTF file for `text()` glyph rendering. One font per compile.
- **fn** (int, optional) — default `$fn` for the whole compile. Per-primitive `fn:` overrides.
- **trace** (string, optional) — dump the evaluator trace to this path. Useful when a nested `for` or `module` isn't producing what you expected.

Real-world showcase — Cyclone-PCB-Factory

For .scad projects big enough that the wasm plugin's 1 GB memory budget gets in the way, precompile natively and hand render-ply a .ply blob directly. `examples/scad/cyclone.typ` in the repo shows the pattern.

The Cyclone PCB Factory CNC mill — a full assembly with MCAD, obiscad, and `standard_parts` under `include <Cyclone.scad>` — compiles to an 11 MB mesh. Too big for the wasm plugin, so it's compiled once by the native harness at `crates/maquette-scad/examples/repro.rs` into a .ply, then rendered by `maquette`:

```
#import "@preview/maquette:0.1.3": render-ply

#let full = read("cyclone-full.ply", encoding: none)
#let openscad-view = (camera: (400, 400, 200), up: (0, 0, 1), fov: 40)
#let shot = (az, el) => render-ply(full,
  openscad-view + (azimuth: az, elevation: el, antialias: 2, zoom: 1.4),
  width: 100%,
)
#shot(35, 20)
#shot(125, 28)
```

The Manifold kernel keeps the whole assembly watertight (0 open edges) — every part renders solid from every angle even where several dozen booleans stack across the MCAD library. To reproduce: obtain `Source_files/` from the upstream repo, drop it into `examples/scad/cyclone-src/`, then

```
cargo run --release --example repro -p maquette-scad
```

which produces `cyclone-full.ply` alongside the .typ.

Language coverage

The .scad evaluator is complete for everything a typical file uses. TL;DR:

Supported (✓) — comments, numbers, bool, string, undef, vectors, ranges, variable assignment (last-wins scope), `let()` expression, member access, indexing, all standard operators, `$fn` / `$t` / `$preview` / `$children`, all 2D + 3D primitives, all transforms (translate, rotate, scale, mirror, resize, multmatrix, color, offset), all booleans (union, difference, intersection), extrusions (`linear_extrude` with twist / scale / slices, `rotate_extrude`), hull, minkowski, projection, list comprehensions, `for`, `if`, `intersection_for`, `function`, `module`, `render`, string ops, math functions, `include` + `use`, `color()` with per-face RGB propagation.

Partial (⊗) — `$fa` / `$fs` are defined so libraries reading them work, but our primitive builder tessellates from `$fn` (or `fn`: on the primitive), not adaptively.

Not supported (✗) — `import()` of external STL/OBJ/DXF (wasm sandbox has no filesystem — pass bytes via `bin`: instead), `surface()` heightmap import, `$vpr` / `$vpt` / `$vpd` / `$vpf` viewport variables (n/a — no viewport at eval time), font-dependent text glyphs beyond basic Latin-1 skeletons.

See `crates/maquette-scad/FIDELITY.md` for the exhaustive per-feature tracker.

Design notes

Why Manifold? — the previous kernel we used (csgrs 0.20.1, BSP-based) left about 20% of faces open on any non-trivial boolean (bores, notches, partial overlaps). Manifold guarantees watertight, correctly-triangulated output. Every mesh this crate emits has zero open edges, verified on the full test corpus.

Why compile in-crate? — a .scad-then-shell-out design would need a filesystem, a working OpenSCAD binary on the host, and IPC across a process boundary. The wasm plugin has none of that. Ingesting .scad text ourselves keeps the whole compile deterministic under Typst’s sandbox — same input, same PDF, cross-platform.

Two wasm modules. — `maquette-scad.wasm` (this crate) compiles geometry to PLY. `maquette.wasm` (the sibling plugin) renders the PLY. They only exchange the PLY blob; you can also send `scadypst(...)`’s output to `maquette-gltf` via a .gltf wrapper if you want PBR shading on procedural geometry.